

Specyfikacja Języka Rusthon++

1. Informacje ogólne

- Autor: Tomasz Nejman
- Przedmiot: Techniki Kompilacji (TKOM)
- Nazwa języka: Rusthon++
- Temat: "typowanie statyczne/silne/domyślnie stałe/przekazywanie przez referencję"

2. Analiza wymagań

2.1. Wymagania funkcjonalne

- **Typowanie i zmienne:** Język jest statycznie i silnie typowany. Pozwala tworzyć zmienne, przypisywać do nich wartości oraz je odczytywać. Semantyka zmiennych zakłada, że są one domyślnie stałe (przypisanie wartości odbywa się raz) z opcją jawnego określenia ich jako mutowalne. Język posiada zdefiniowane zakresy widoczności zmiennych z mechanizmem przykrywania (shadowing) wewnątrz bloków.
- **Typy liczbowe i operacje:** Język obsługuje podstawowe typy danych liczbowych (całkowitoliczbowe `int` i zmiennoprzecinkowe `float`). Zapewnia obsługę operacji matematycznych o różnym priorytecie wykonania oraz pozwala na użycie nawiasów do grupowania wyrażeń. Analizator leksykalny wspiera znaki separacji w liczbach (np. `1_000_000`).
- **Typ logiczny i operacje:** Język obsługuje typ logiczny (`bool`). Zapewnia obsługę operacji logicznych o różnym priorytecie wykoanania oraz pozawala na użycie nawiasów do grupowania wyrażeń.
- **Typ znakowy i tekstowy:** Język obsługuje typ znakowy (`char`) oraz tekstowy (`string`). Zdefiniowane są operacje na ciągach znaków, w tym minimalnie konkatenacja. Ciągi tekstowe mogą zawierać dowolne znaki, włącznie z wyróżnikami stringa (np. znak cudzysłowu lub apostrofu poprzedzony znakiem ucieczki), znakami nowej linii i tabulacji.
- **Komentarze:** Język obsługuje komentarze jednolinijkowe.
- **Sterowanie przepływem:** Język obsługuje instrukcje warunkowe (`if`, `elseif`, `else`) oraz instrukcję pętli (`while`, `break`, `continue`).
- **Funkcje:** Język daje możliwość definiowania oraz wołania własnych funkcji, które posiadają własne zmienne lokalne. Funkcje mogą być wywoływane rekursywnie.
- **Przekazywanie argumentów:** Semantyka języka definiuje przekazywanie argumentów do funkcji. Domyślnie wartości są referencjami, natomiast wdrożono jawną możliwość przekazywania zmiennych przez wartość za pomocą dedykowanego słowa kluczowego `copy`.
- **Obsługa błędów:** Zaimplementowano mechanizm obsługi i zgłaszania błędów wykonania. Język wspiera mechanizm wyjątków (słowa kluczowe `throw`, `catch`).
- **Typ niestandardowy:** Język wspiera homogeniczne kolekcje (tablice) z przypisanym zestawem wbudowanych operatorów (m. in. moc zbioru, mapowanie, filtrowanie).

2.2. Wymagania niefunkcjonalne

- Analiza kodu podzielona jest na moduły:
 - lekser
 - parser
 - interpreter
- Każdy z modułów jest napisany w C++.
- Interpreter jest aplikacją konsolową (CLI) i umożliwia wczytanie kodu z pliku lub wprowadzanie w trybie interaktywnym.
- Role modułów wyglądają następująco:
 - lekser - wczytuje żywy tekst / przyjmuje strumień znaków (kodu) i wyprowadza strumień tokenów
 - parser - otrzymuje reprezentację tokenową z leksera i buduje drzewo AST
 - interpreter - dokonuje statycznej analizy i wylicza stałe, następnie przemierza drzewo wykonując program.

3. Formalna składnia (gramatyka)

3.1. Symbole terminalne (Lekser)

COMMENT	::= r"#.*"
INT_LITERAL	::= r"[0-9]+(_[0-9]+)*"
FLOAT_LITERAL	::= r"[0-9]+(_[0-9]+)*\.[0-9]+(_[0-9]+)*"
STRING_LITERAL	::= r'"[^\\]*(\\.[^\\])*"'
CHAR_LITERAL	::= r"'[^\\]' '\\[nr\\t'\"\\]'"
BOOL_LITERAL	::= r"\b(true false)\b"
TYPE_INT	::= r"\bint\b"
TYPE_FLOAT	::= r"\bfloat\b"
TYPE_CHAR	::= r"\bchar\b"
TYPE_STRING	::= r"\bstring\b"
TYPE_BOOL	::= r"\bbool\b"
TYPE_VOID	::= r"\bvoid\b"
KW_IF	::= r"\bif\b"
KW_ELSEIF	::= r"\belseif\b"
KW_ELSE	::= r"\belse\b"
KW_WHILE	::= r"\bwhile\b"

```

KW_DO      ::= r"\bdo\b"
KW_DONE    ::= r"\bdone\b"
KW_BREAK   ::= r"\bbreak\b"
KW_CONTINUE ::= r"\bcontinue\b"
KW_RET     ::= r"\bret\b"

OP_AND     ::= r"\band\b"
OP_OR      ::= r"\bor\b"
OP_NOT     ::= r"\bnot\b"

KW_THROW   ::= r"\bthrow\b"
KW_TRY     ::= r"\btry\b"
KW_CATCH   ::= r"\bcatch\b"
KW_FINALLY ::= r"\bfinally\b"
KW_EXCEPTION ::= r"\bexception\b"

KW_DEF     ::= r"\bdef\b"
KW_VAR     ::= r"\bvar\b"
KW_COPY    ::= r"\bcopy\b"
KW_CAST    ::= r"\bcast_to\b"
KW_MUT     ::= r"\bmust\b"

OP_MAP     ::= r"->"
OP_EQ      ::= r"=="
OP_NEQ     ::= r"!="
OP_GEQ     ::= r">="
OP_LEQ     ::= r"<="

OP_ADD_ASSIGN ::= r"\+="
OP_SUB_ASSIGN ::= r"\-="
OP_MUL_ASSIGN ::= r"\*="
OP_DIV_ASSIGN ::= r"\/="

OP_CONCAT  ::= r"\+\+"
OP_DIFF    ::= r"--"

OP_ASSIGN  ::= r"="
OP_PLUS    ::= r"\+"
OP_MINUS   ::= r"- "
OP_MUL     ::= r"\*"
OP_DIV     ::= r"/"
OP_MOD     ::= r"%"
OP_LEN     ::= r"\$"
OP_FILTER  ::= r"\?"
OP_GT      ::= r">"
OP_LT      ::= r"<"
OP_REV     ::= r "@"

COLON      ::= r":"
COMMA      ::= r","
LPAREN     ::= r\"(\"
RPAREN     ::= r\")"
LBRACKET   ::= r\"["
RBRACKET   ::= r\"]"

IDENTIFIER ::= r"[a-zA-Z][a-zA-Z0-9_]*"

NEWLINE    ::= r"\r?\n"
WS         ::= r"[ \t]+"

```

Tokeny `COMMENT` i `WS` oczywiście są odrzucane i nie trafiają do parsera.

3.2. Gramatyka EBNF (Parser)

```

Program      ::= StatementList

```

```

Program      ::= StatementList
StatementList ::= { Statement NEWLINE }
Statement    ::= FunctionDef
               | VarDecl
               | ConstDecl
               | IfStmt
               | WhileStmt
               | LoopControl
               | ReturnStmt
               | ThrowStmt
               | TryCatchStmt
               | ExpressionStmt

FunctionDef  ::= "def" ReturnType IDENTIFIER "(" [ ParamList ] ")" "do" StatementList "done"
ReturnType  ::= Type | "void"
ParamList   ::= Param { "," Param }
Param       ::= [ ParamModifier ] Type IDENTIFIER
ParamModifier ::= "copy" | "mut"

VarDecl     ::= "var" Type IDENTIFIER "=" Expression
ConstDecl   ::= Type IDENTIFIER "=" Expression
ExceptionDecl ::= "exception" IDENTIFIER

IfStmt      ::= "if" "(" Expression ")" "do" StatementList "done"
               { "elseif" "(" Expression ")" "do" StatementList "done" }
               [ "else" "do" StatementList "done" ]

WhileStmt    ::= "while" "(" Expression ")" "do" StatementList "done"
LoopControl  ::= "break" | "continue"
ReturnStmt   ::= "ret" [ Expression ]
ThrowStmt    ::= "throw" IDENTIFIER Expression
TryCatchStmt ::= "try" "do" StatementList "done"
               "catch" IDENTIFIER [ IDENTIFIER ] "do" StatementList "done" { "catch" IDENTIFIER [ IDENTIFIER ] "do" Stateme
               [ "finally" "do" StatementList "done" ]

Type         ::= BaseType | ArrayType
BaseType     ::= "int" | "float" | "char" | "string" | "bool"
ArrayType    ::= "[" Type "]"

ExpressionStmt ::= Expression
Expression      ::= LogicalOrExpr [ AssignmentOp Expression ]
AssignmentOp    ::= "=" | "+=" | "-=" | "*=" | "/="
LogicalOrExpr   ::= LogicalAndExpr { "or" LogicalAndExpr }
LogicalAndExpr  ::= EqualityExpr { "and" EqualityExpr }
EqualityExpr    ::= RelationalExpr { ( "==" | "!=" ) RelationalExpr }
RelationalExpr  ::= AdditiveExpr { ( "<" | "<=" | ">" | ">=" ) AdditiveExpr }
AdditiveExpr    ::= MultiplicativeExpr { ( "+" | "-" | "++" | "--" ) MultiplicativeExpr }
MultiplicativeExpr ::= UnaryExpr { ( "*" | "/" | "%" ) UnaryExpr }
UnaryExpr      ::= ( "-" | "not" | "@" | "$" ) UnaryExpr
               | CastExpr
CastExpr       ::= AccessExpr { "cast_to" Type }
AccessExpr     ::= PrimaryExpr { "[" Expression "]" | "->" IDENTIFIER | "?" IDENTIFIER }
PrimaryExpr    ::= IDENTIFIER [ "(" [ ArgumentList ] ")" ]
               | Literal
               | "(" Expression ")"
               | ArrayLiteral

ArgumentList  ::= Expression { "," Expression }
ArrayLiteral  ::= "[" [ ArgumentList ] "]"
Literal       ::= INT_LITERAL
               | FLOAT_LITERAL
               | STRING_LITERAL
               | CHAR_LITERAL
               | BOOL_LITERAL

```

4. Semantyka i konstrukcje językowe

Aby zilustrować zakładaną funkcjonalność przed szczegółowym opisem, poniżej zamieszczam przykład wykorzystania kodu:

```
# predykat do filtrowania
def bool is_adult(copy int age) do
    ret age >= 18
done

# funkcja modyfikująca referencję i zwracająca podzbiór
def [int] process_users(mut int valid_count, [int] ages) do
    [int] adults = ages ? is_adult      # filtrowanie kolekcji względem predykatu
    valid_count = $adults              # moc kolekcji
    ret adults
done

var int adult_count = 0
[int] raw_ages = [12, 19, 25, 15, 30]

# przekształcenie zmiennej mutowalnej przez referencję
[int] processed = process_users(adult_count, raw_ages)
# Wartości po wykonaniu:
#   processed == [19, 25, 30]
#   adult_count == 3
```

4.1. System typów i zmienne

4.1.1. Deklaracje i przekazywanie

Zmienne domyślnie inicjowane są jako stałe. Aby zadeklarować zmienną mutowalną, używa się słowa kluczowego `var`. Typ zmiennej jest statyczny i określa się go jawnie przed identyfikatorem zmiennej/stałej. Typ zmiennej/stałej jest niezmienny. Deklaracja zmiennej nie może wystąpić bez przypisania wartości.

Operator przypisania `=` zawsze wykonuje głęboką kopię wartości.

Przykłady deklaracji i inicjalizacji:

```
int max_users = 10      # domyślna stała
var int counter = 0     # zmienna mutowalna
```

Przykład poprawny, zmiana wartości w ramach jednego typu:

```
var string message = "hello"
message = "world"
```

Przykład niepoprawny zmiany zmiennej (stałej):

```
var int age = 20
age = "abc"      # błąd - niezgodność typów

int id = 123
id = 456         # błąd - próba modyfikacji stałej
```

Przykład kopiowania przez operator `=`:

```
int original_counter = 5
var int counter_copy = original_counter # counter_copy to nowa niezależna zmienna
counter_copy = 10                      # counter_copy jest zmienną, mimo że original_counter było stałą
```

Referencje w języku Rusthon++ istnieją tylko w przypadku przekazywania argumentów wywołań funkcji. Sposoby przekazywania argumentów:

1. Stała referencja (domyślna): brak modyfikatora przed typem parametru (np. `[int] nums`).
2. Mutowalna referencja: słowo kluczowe `mut` przed typem parametru (np. `mut int valid_count`). Aby wywołanie było poprawne, argument przekazywany do funkcji musi być mutowalny.
3. Przekazywanie przez wartość: słowo kluczowe `copy` (np. `copy int age`).

4.1.2. Podstawowe typy danych

- Język dostarcza typ całkowitoliczbowy ze znakiem `int`. Legalne wartości typu `int` to zakres od `-2147483648` do `2147483647`. Język wspiera znak separacji `_` dla łatwiejszego wprowadzania dużych literałów. np. `int x = 1_000_000`
- Język dostarcza typ zmiennoprzecinkowy pojedynczej precyzji ze znakiem `float`. Granice wartości dla typu `float` (nie podaję więcej cyfr znaczących z powodu utraty precyzji):
 - Największa liczba dodatnia: `3.4028235e+38`
 - Najmniejsza liczba ujemna: `-3.4028235e+38`
 - Najmniejsza liczba dodatnia: `1.17549435e-38`
 - Największa liczba ujemna: `-1.17549435e-38`
- Język dostarcza typ logiczny `bool` o możliwych wartościach `true` lub `false`.

4.1.3. Typy znakowe, tekstowe i komentarze

Typ znakowy `char` zapisywany jest w pojedynczych apostrofach, natomiast ciągi znaków `string` w podwójnych cudzysłowach. Język wspiera znaki ucieczki z użyciem ukośnika `\`, pozwalając na wprowadzenie nowej linii `\n`, tabulacji `\t`, wyróżników tekstu `\"`, `\'` oraz ukośnika `\\`. Typ złożony `string` jest jednoznaczny z typem `[char]` (tablica `char`), a zatem możliwe są do użycia na nim dowolne operatory kolekcji.

```
char letter = 'A'
char quote = '"'
string message = "He said: \"Hello World!\"\n"
```

```
string word1 = "Hello,"
string word2 = "World"
string word3 = "!\n"

string result = word1 ++ @word2 ++ word3 # result == "Hello, dlrow!\n"
```

Komentarze jednolinijkowe rozpoczynają się znakiem `#`. Wszystko na prawo od tego znaku w danej linii jest ignorowane.

```
int a = 5 # To jest poprawny komentarz w linii z kodem
```

4.1.4. Konwersja typów (casting)

Język charakteryzuje się silnym typowaniem i nie wykonuje niejawnych konwersji. Aby dokonać konwersji należy użyć operatora kluczowego `cast_to`, a po nim typu na który się rzutuje. Służy ono do konwersji pomiędzy kompatybilnymi typami (np. `int` na `float`) i wartościami (np. typ `string` o wartości `"100"` na `int`). Operacja tworzy nową, mutowalną wartość i dba o zachowanie jej sensu matematycznego/logicznego. W przypadku niemożności wykonania konwersji w czasie działania programu, zgłaszany jest wyjątek wykonania. Operator `cast_to` ma najwyższy priorytet (nie licząc nawiasów) i wiąże silniej niż operatory arytmetyczne.

Przykład poprawnej i niepoprawnej konwersji:

```
float a = 3 cast_to float # a == 3.0
int b = (3.20 + 3.90) cast_to int # b == 7

float c = "abc" cast_to float # błąd - brak przejścia z typu 'string' do 'float' dla wartości "abc"
```

Zasady konwersji dla podstawowych typów przedstawia poniższa tabela: (`cast_to` oczywiście działają jeżeli rzutowanie jest z typu `T` do `T` po prostu kopiując wartość)

Z\Do	int	float	char	bool	string
int	N/A	dopisuje się <code>.0</code>	kod Unicode, np. <code>65 \$→\$ 'A' **</code>	<code>false</code> dla <code>0</code> , <code>true</code> dla reszty	reprezentacja tekstowa np. <code>100 \$→\$ "100"</code>
float	obcięcie części dziesiętnej, w stronę zera	N/A	niedozwolone	<code>false</code> dla <code>0.0</code> , <code>true</code> dla reszty	reprezentacja tekstowa, np. <code>3.14 \$→\$ "3.14"</code>
char	wartości Unicode, np. <code>'A' \$→\$ 65</code>	niedozwolone	N/A	niedozwolone	znak jako tekst: <code>'a' \$→\$ "a"</code>
bool	<code>1</code> dla <code>true</code> lub <code>0</code> dla <code>false</code>	<code>1.0</code> dla <code>true</code> lub <code>0.0</code> dla <code>false</code>	niedozwolone	N/A	<code>"true"</code> lub <code>"false"</code>
string	parsowanie tekstu, wyjątek przy błędzie	parsowanie tekstu, wyjątek przy błędzie*	zwraca znak, jeśli długość tekstu to dokładnie 1, w przeciwnym razie wyjątek	<code>true</code> jeżeli tekst jest niepusty, <code>false</code> w przeciwnym przypadku	N/A

`int` * "1" to też legalny "float", nie potrzebna jest część przecinkowa przy konwersji ze `string` ** legalne wartości to zakres od `0x0` - `0x10FFFF`, z wyłączeniem przedziału `0xD800` - `0xDFFF`.

4.2. Operatory i wyrażenia

4.2.1 Priorytet operatorów

Priorytet jest od największego do najmniejszego. W przypadku każdego priorytetu, wyższość ma operator po lewej.

1. `()` - nawiązywanie, łączność lewostronna
2. `[]`, `->`, `?` - dostęp i transformacje, łączność lewostronna
3. `cast_to` - rzutowanie typów, łączność lewostronna
4. `-`, `not`, `@`, `$` - operatory unarne, łączność prawostronna
5. `*`, `/`, `%` - multiplikatywne, arytmetyczne, łączność lewostronna
6. `+`, `-`, `++`, `--` - addytywne, arytmetyczne i kolekcje, łączność lewostronna
7. `<`, `<=`, `>`, `>=` - relacyjne (nierówności), łączność lewostronna
8. `==`, `!=` - relacyjne (równości), łączność lewostronna
9. `and` - logiczne AND, łączność lewostronna
10. `or` - logiczne OR, łączność lewostronna
11. `=`, `+=`, `-=`, `*=`, `/=` - przypisanie, łączność prawostronna

4.2.2. Operatory przypisania

Każdy operator przypisania wymaga operandów tego samego typu. Każdy operator przypisania arytmetycznego (każdy tutaj poza `=`) wymaga operandów typu `int` lub `float`.

1. Przypisanie standardowe `=`
2. Przypisanie z dodawaniem `+=`
3. Przypisanie z odejmowaniem `-=`
4. Przypisanie z mnożeniem `*=`
5. Przypisanie z dzieleniem `/=`

4.2.3. Operacje arytmetyczne

Każdy operator wymaga dwóch operandów. Oba muszą być tego samego typu i być `int` lub `float`, z wyjątkiem modulo, które przyjmuje tylko `int`.

1. Dodawanie `+`
2. Odejmowanie `-`
3. Mnożenie `*`
4. Dzielenie `/`
5. Modulo `%`

4.2.4. Operacje logiczne i relacyjne

1. `not` - jeden operand typu wyrażenie logiczne, prawostronny
2. `and` - dwa operandy typu wyrażenie logiczne
3. `or` - dwa operandy typu wyrażenie logiczne

Stosuje się leniwą ewaluację wyrażeń logicznych, np.

```
if (y != 0 and x / y > 5) do # jeżeli y==0, to drugi warunek nie jest sprawdzany
    # blok kodu
done
```

1. Większy `>`
2. Mniejszy `<`
3. Większy lub równy `>=`
4. Mniejszy lub równy `<=`
5. Równy `==`
6. Różny `!=`

Każdy operator wymaga dwóch operandów. Oba muszą być tego samego typu. Operatory nierówności wymagają typu `int` lub `float`. Operatory równości `==` oraz `!=` działają dla dowolnego typu i porównują wartość.

4.2.5. Operatory specjalne

1. Nawiasy `()`

Do sztucznego zwiększania priorytetu operacji lub wywołania funkcji.

4.2.6. Zachowanie w nietypowych sytuacjach

1. Typ `int` W przypadku over- lub underflow wartość "przekręca się".

```
var int x = 2147483647
x += 1          # x == -2147483648
```

Dzielenie przez zero kończy się rzuceniem wyjątku.

2. Typ float

(głównie standardowe zachowania typu float w języku C++)

1. przekroczenie górnego limitu wartości: `inf`
2. przekroczenie dolnego limitu wartości: `-inf`
3. dzielenie przez zero jest legalne: `1.0 / 0.0 == inf`, `-1.0 / 0.0 == -inf`
4. dla operacji niemożliwych do wyznaczenia/nieokreślonych: `NaN`
5. Na wartości `NaN` można wykonywać operacje matematyczne, ale wszystkie dają `NaN` jako wynik
6. Na wartościach `inf` oraz `-inf` większość operacji nie ma efektu. Przypadki szczególne to np. a. `inf / inf == NaN` b. `+ / inf == 0.0`

4.3. Złożone struktury danych

4.3.1. Kolekcje i ich operatory

Język wspiera kolekcje homogeniczne (tablice). Kolekcje mogą być tworzone w locie z użyciem nawiasów kwadratowych `[]`. Operator przypisania domyślnie przekazuje kolekcje przez głęboką kopię. Pozycje w kolekcjach indeksowane są od `0`.

Zdefiniowane operatory dla kolekcji:

1. `[]` - dostęp do elementu
2. `++` - konkatenacja dwóch kolekcji
3. `--` - różnica dwóch kolekcji
4. `$` - moc kolekcji (liczba elementów)
5. `@` - odwrócenie kolekcji (operator unarny)
6. `?` - filtrowanie względem predykatu (oczekuje identyfikatora funkcji o sygnaturze `bool foo(T)`)
7. `->` - mapowanie względem funkcji (oczekuje identyfikatora funkcji o sygnaturze `T1 foo(T2)`, gdzie `T2` to typ w kolekcji)

Przykłady użycia operatorów:

```
[int] nums1 = [1, 2, 3]
[int] nums2 = [3, 4, 5]

[int] combined = nums1 ++ nums2    # [1, 2, 3, 3, 4, 5]
[int] sub = nums1 -- nums2         # [1, 2]
[int] count = $nums1               # 3
[int] reversed = @nums             # [3, 2, 1]

# predykat do filtrowania
def bool isBig(copy int x) do
    ret x > 3
done

[int] big_nums = nums2 ? isBig      # [4, 5]

# funkcja do mapowania
def int makeDouble(copy int x) do
    ret x * 2
done

[int] nums_double = nums1 -> makeDouble # [2, 4, 6]
```

* operator mapowania `->` pozwala na mapowanie funkcją z dowolnego typu `T2` na dowolny inny `T1`, np.

```
def bool isSmall(copy int x) do
    ret x < 0
done

[int] nums = [-2, -1, 0, 1, 2]
[bool] is_small_arr = nums -> isSmall    # is_small_arr == [true, true, false, false, false]
```

4.4. Przepływ sterowania

4.4.1. Instrukcje sterujące

Język wspiera instrukcje warunkowe (słowa kluczowe: `if`, `elseif`, `else`) oraz pętle (`while`). Ich bloki otwierane otwierane są słowem kluczowym `do`, wcięcie jest dowolne. Koniec bloku wyznaczany jest poprzez słowo kluczowe `done`. Dozwolona jest jedna instrukcja na linię. Podczas pętli `while` można ominąć pozostałe operacje w danym przejściu, używając słowa kluczowego `continue`. Można też przerwać pętlę słowem kluczowym `break`. `if`, `elseif` i `while` wymagają konkretnie typu `bool` (silne typowanie).

Przykład instrukcji warunkowych:

```
var int x = 3
if (x < 1) do
    x = x + 1
done elseif (x < 5) do
    x = x + 2
done else do
    x = x + 3
done
```

Przykład pętli:

```
var int x = 5
while (x < 10) do
    x = x + 1
done
```

Przykład pominięcia i przerywania pętli:

```
var int x = 5
while (x < 100) do
    x = x + 1
    if (x % 17 == 0) do
        continue
    done
    if (x == 19) do
        break
    done
done
```

4.4.2. Funkcje, zakres widoczności i przykrywanie zmiennych

Funkcje definiowane są słowem kluczowym `def`, po którym następuje zwracany typ, identyfikator oraz lista argumentów w nawiasie. Słowo `ret` zwraca wartość z funkcji. Brak wartości zwracanej oznaczany jest `void`, a słowo kluczowe `ret` nie jest wtedy wymagane. Ciało funkcji również rozpoczyna się słowem kluczowym `do` i kończy `done`. Jeżeli funkcja deklaruje zwracany typ i nie jest nim `void`, a interpreter dojdzie do końca funkcji i nie znajdzie słowa `ret` rzucany jest błąd.

Funkcje mogą być przeciążane. Interpreter rozróżnia je na podstawie liczby, typów, a także mutowalności argumentów. Wbudowane funkcje języka podlegają tym samym zasadom i mogą być przykrywane lub przeciążane przez użytkownika.

Funkcje zawsze zwracają kopię.

Funkcje mogą być wywoływane rekursywnie. Limit rekursji to 100 wywołań.

Funkcje mogą znajdować się tylko na najwyższym poziomie pliku.

Język wspiera przykrywanie zmiennych (shadowing). Zmienne deklarowane wewnątrz bloku (np. instrukcji warunkowej) mogą mieć taką samą nazwę jak zmienne w zakresie zewnętrznym. Po wyjściu z bloku przywracany jest dostęp do oryginalnej zmiennej z zachowaniem jej wcześniejszego stanu.

Przykład funkcji:

```
def int add(copy int a, copy int b) do
    ret a + b
done
```

Wersja funkcji przyjmująca mutowalną referencję i nic nie zwracająca:

```
def void process(mut int a, mut int b) do
    a += b
done
```

Wersja funkcji przyjmująca niemutowalną referencję:


```
def int addTen(int a) do
    ret a + 10
done
```

Wersja funkcji przyjmująca dwie zmienne przekazywane przez wartość:

```
def int add(copy int a, copy int b) do
    ret a + b
done
```

Przeciążanie funkcji o tej samej nazwie:

```
def int add(copy int a, copy int b) do
    ret a + b
done

def int add(copy int a, copy int b, copy int c) do
    ret a + b + c
done
```

Przykład mechanizmu przykrywania zmiennych:

```
var int x = 5
if (x > 5) do
    int x = 10    # shadowing - 'x' w tym bloku jest stałą o wartości 10
    # x = 11      # błąd - 'x' w tym bloku jest stałą
done
x = 20           # po wyjściu z bloku 'x' ponownie wynosi 5 i jest mutowalny
```

4.4.3. Mechanizm wyjątków

Język dostarcza mechanizm rzucania i łapania wyjątków. Można łapać tylko wyjątki z rodziny `RuntimeException`, gdyż błędy niższego poziomu nie pozwalają na sensowną interpretację czegokolwiek. Słowa kluczowe: `throw`, `try`, `catch`, `finally`. Użytkownik może tworzyć własne wyjątki i może wykorzystywać istniejące, wbudowane. Aby złapać dowolny wyjątek, a nie konkretny można użyć słowa `exception` w konstrukcji `catch exception`.

Wyjątki rzucane przez użytkownika mogą przekazywać wiadomości przez wiadomość (`string`). Wyjątek niezłapany aż do najwyższego poziomu oznacza zakończenie programu.

Przykłady:

```
var int x = 5
int y = 0
try do
    x = x / y
done catch div_by_zero_exception e do
    write(e)    # "div_by_zero_exception"
done
```

```
exception my_exception
throw my_exception "This is a message in my exception"
```

```
var int x = 0
try do
    x += read() cast_to int
done catch bad_cast_exception e do
    write(e)    # przykładowa wiadomość "bad_cast_exception: cannot cast string "apple" to int"
done finally do
    x = x + 1
done
```

```
def int safeDivide(copy int a, copy int b) do
  if (b == 0) do
    throw div_by_zero_exception "cannot divide by zero"
  done
  ret a / b
done
```

```
try do
  dangerousFunction()
done catch div_by_zero_exception e do
  # blok kodu
done catch bad_cast_exception e do
  # blok kodu
done catch index_out_of_bounds_exception e do
  # blok kodu
done
```

```
try do
  throw exception "this is my exception message"
done catch exception e do
  write(exception)          # "exception: msg: "this is my exception message""
done
```

4.5. Wbudowane wejście i wyjście (I/O)

Język dostarcza dwie funkcje `read` i `write`, o sygnaturach `string read()` i `void write(T)`. Funkcja `read` jest bezargumentowa i wczytuje dane użytkownika wpisane w terminalu; zwraca zawsze typ `string`. Funkcja `write` może przyjąć jeden argument, dowolnego typu, ponieważ użytkownik nie może dfiniować własnych typów zachowanie zawsze jest znane i nic nie zwraca. Funkcja `write` nie przesuwaa kursora automatycznie do następnej linii, należy używać manualnie `'\n'`.

Przykłady użycia:

```
string name = read()          # wprowadzamy "Jan"
write("Hello, " ++ name ++ '\n') # >> "Hello, Jan"
```

```
[int] nums = [1,2,3,4]
var int i = 0
while (i < 5) do
  write(nums[i])
  write(' ')
  i += 1
done

# >> 1 2 3 4
```

```
[string] names = ["Anna", "Bob", "Charles"]
write(names)      # >> [ "Anna", "Bob", "Charles" ]
```

5. Raportowanie błędów i diagnostyka

Stosuje się strategię "fail-fast". Oznacza to, że po napotkaniu błędu na dowolnym etapie, program kończy się błędem.

5.1. Typy zgłaszanych błędów

5.1.1. Błędy leksykalne

Wykrywane przez lekser podczas tokenizacji.

- Przyczyny: Nierozpoznany znak, niezamknięty cudzysłów stringa, niezamknięty apostrof znaku
- Reakcja: Rzucenie wyjątku `LexerException` z podaniem niepoprawnego znaku i jego pozycji.

5.1.2. Błędy składniowe

Wykrywane przez parser podczas budowania drzewa AST z tokenów.

- Przyczyny: Brakujący nawias, nieoczekiwany token, brak słowa kluczowego, np. `do` po warunku instrukcji `if` .
- Reakcja: Rzucenie wyjątku `SyntaxException` / `ParseException` wskazującego token aktualnie napotkany i oczekiwany token.

5.1.3. Błędy semantyczne

Wykrywane przez interpreter podczas analizy semantycznej. Wyjątki rzucane są z rodziny `SemanticException` .

Przykłady:

Przyczyna	Rzucony wyjątek
Niezgodność typów (np. próba użycia operatora <code>+</code> na typie <code>int</code> i <code>string</code> bez rzutowania)	<code>incompatible_type_exception</code>
Próba przypisania nowej wartości do zmiennej zadeklarowanej jako stała lub proba zmiany niemutowalnej referencji	<code>modify_constant_exception</code>
Użycie niezadeklarowanej zmiennej/funkcji	<code>unknown_identifier_exception</code>
Niepoprawna liczba argumentów podczas wywołania funkcji	<code>unmatched_function_overload_exception</code>
Nieudane rzutowanie literału, np. <code>"apple" cast_to int</code>	<code>bad_literal_cast_exception</code>

5.1.4. Błędy wykonania

Wykrywane przez interpreter podczas wykonania kodu. Wyjątki rzucane są z rodziny `RuntimeException` .

Przykłady:

Przyczyna	Rzucony wyjątek
Błędy wbudowanych funkcji matematycznych (np. dzielenie przez 0)	<code>div_by_zero_exception</code>
Wyjście poza zakres tablicy	<code>index_out_of_bounds_exception</code>
Nieudane rzutowanie (np. <code>x cast_to int</code> , gdzie <code>x</code> jest typu <code>string</code> i ma wartość <code>apple</code>)	<code>bad_cast_exception</code>

5.2. Przykładowe komunikaty o błędzie

Błędy leksykalne, składniowe i semantyczne podają po prostu lokalizację błędu i przyczynę. Błędy wykonania podają też traceback, aby ułatwić debugowanie.

Błąd leksykalny:

```
Lexical Error at line 8, column 22:
Unterminated string literal.
```

Błąd składniowy (brakujący token):

```
Syntax Error at line 4, column 15:
Unexpected token 'IDENTIFIER' (value: "x"). Expected 'do'.
```

Błąd semantyczny (próba zmiany stałej):

```
Semantic Error at line 12, column 11:
Cannot mutate 'max_users'. Variable was declared as constant.
```

Błąd wykonania (wyjście poza zakres tablicy):

```
def int findSalary([int] sal, copy int id) do
    ret sal[id]
done

[int] salaries = [1000, 2000, 3000, 4000]
int salary = findSalary(salaries, 6)
```

```
Runtime Error at line 2, column 12:
Index "6" out of bounds of array "sal", which is of size "4".
Caused by:
Function call "findSalary" at line 6, column 13 at top level
```

Błąd wykonania (niezłapany wyjątek)

```
int x = 3
if (x > 0) do
    throw index_out_of_bounds_exception
done
```

```
Runtime Error at line 3, column 5:
Unhandled exception: "index_out_of_bounds_exception" at top level
```

6. Narzędzia i cykl życia

6.1. Narzędzia (komponenty)

Proces analizy i wykonania kodu

- Interpreter żąda od parsera jednej jednostki wykonawczej
- Parser czyta strumień tokenów, aż sformułuje pełną, logiczną instrukcję, np. przypisanie
- Na żądanie parsera lexer czyta strumień znaków i przygotowuje po jednym tokenie (czyta z pliku, argumentu lub terminala w przypadku sesji interaktywnej)
- Parser buduje poddrzewo dla tej instrukcji
- Interpreter odwiedza poddrzewo i je wykonuje
- Po wykonaniu poddrzewo może zostać usunięte, bo wszystkie identyfikatory i ich stan został zapisany w tabeli identyfikatorów

Wyjątkami są funkcje i bloki, które muszą być wczytane w całości naraz.

Przykrywanie zmiennych

Tabela zmiennych jest dwuwymiarowa (1-zmienne, 2-zakresy). Gdy tworzony jest nowy blok, np. ciało funkcji zostaje dodany nowy wymiar (efektywnie push na stos). Interpreter szukając identyfikatora przegląda tabelę od najwyższej pozycji (najnowszego zakresu). Gdy blok się kończy, cały wymiar zostaje odrzucony, przez co garbage collector nie jest potrzebny.

6.2. Sposób uruchomienia

1. Wczytanie kodu z pliku (dowolny plik tekstowy):

```
interpreter script.txt
```

2. Wczytanie kodu z argumentu

```
interpreter -a "int x = 5
write(x)"
5
```

3. Interaktywna sesja:

```
interpreter
>> var int x = 5
>> x += 5
>> write(x)
>> 10
```

Zakończenie sesji: `exit()`

Wbudowana funkcja `exit()` służy do zakończenia działania programu i może być również użyta w kodzie. Przyjmuje jeden argument (`int`) i zwraca go jako kod wyjścia programu lub zero argumentów i zwraca `0` .

7. Przykłady użycia kodu

(test akceptacyjny na później)

```

def bool is_valid_salary(int salary) do
  ret salary >= 3000
done

def int apply_bonus(copy int salary) do
  int base_bonus = 500
  ret salary + base_bonus
done

def [int] process_salaries([string] raw_data, mut int error_count) do
  var [int] parsed_salaries = []
  var int i = 0

  while (i < |raw_data|) do
    try do
      int val = raw_data[i] cast_to int
      parsed_salaries = parsed_salaries ++ [val]
    done catch exception do
      error_count += 1
      write("Parsing error: " ++ raw_data[i] ++ "\n")
    done

    i += 1
  done

  [int] valid_salaries = parsed_salaries ? is_valid_salary
  [int] final_salaries = valid_salaries -> apply_bonus

  ret final_salaries
done

[string] input_data = ["4000", "2500", "invalid_data", "8000", "-500", "3200"]
var int total_errors = 0

[int] results = process_salaries(input_data, total_errors)

write("Err count: " ++ total_errors cast_to string ++ "\n")

var int sum = 0
var int idx = 0

while (idx < $results) do
  int current = results[idx]

  if (current > 5000) do
    int extra_bonus = 1000
    sum += (current + extra_bonus)
  elseif (current == 4500) do
    int current = 0
    sum += current
  else do
    sum += current
  done

  idx += 1
done

write("Sum: ")
write(sum)
write("\n")

```